QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **Vegetable Chicken Cubes***
Team Repo URL	*https://github.com/Angelinetzx0210/CloThinks*
Project Board URL	*https://jolinchong10.atlassian.net/jira/software/projects/KAN/boards/1?jql=assignee+%3D+empty&atlOrigin=eyJpIjoiNjljYjFiZWEwODQ0NGQ5N2E4NGRlZGZjZTUyNTQ3MTUiLCJwIjoiaiJ9*
Live Deployment URL (optional for preliminary round)	*https://clothinks-system.space.z.ai*

Objective:

Ensure that CloThinks accurately tracks inbound/outbound stock for clothing SMEs, reduces textile waste, predicts overstock, and provides reliable style & size matching for customers using an AI recommendation engine. with CI/CD checkpoints.]

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing with specific user requirements via the " **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for those specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- ***Stock In/Out Tracking:*** *Record deliveries, returns, sales, damaged items, and seasonal transfers.*
- ***Waste Reduction Logic:*** *Flag slow-moving items, suggest discounting or repurposing.*
- ***Overstock Utilization:*** *Generate style suggestions from existing unsold stock.*
- ***Customer Personalised Style & Size Matcher****: Input customer preferences & body measurements to AI to suggest matching items.*

1.2 Out-of-Scope

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Payment gateway integration
- Multi-store warehouse logistics (only single SME per instance)
- Fabric quality lab testing

2. Risk Assessment & Mitigation Strategy

*[e.g., Quality Assurance risks are anticipatory. So, identify the technical risk associated with your application requirements and architecture. Now, the risk needs to be evaluated using the 5*5 Risk Assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	**Likelihood (1–5)**	**Severity (1–5)**	**Risk Score (L×S)**	**Mitigation Strategy**	**Testing Approach**
AI suggests sold-out or mismatched size (hallucination)	*4*	*5*	*20 (Severity)*	*Perform real-time stock check and size filtering before recommendation*	*Use cases to mock run customer input and compare to AI output against the inventory database*
Stock in/out mismatch	*3*	*4*	*15 (High)*	*Implement idempotent APIs and database transactions with rollback mechanisms*	*Stimulate concurrent purchase and cancellation scenarios, and verify final stock consistency*
Overstock suggestions are causing a lack of diversity (repetition)	*3*	*3*	*9 (Medium)*	*Introduce diversity scoring and randomization*	*Test across multiple user profiles and compare variation in outputs*
Inaccurate size recommendation(+/-1 size variance)	*4*	*4*	*16(High)*	*Use range-based sizing (eg, M fits 36-38) and feedback-based refinement*	*Test borderline measurements to ensure correct size recommendations*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating, and thorough testing is required*
16 – 25	*Critical*	*Priority is Highest. Need an extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data, and the rules/thresholds that you are incorporating to govern your testing phases.

- **Unit Test**
  - ***Scope:*** *Stock update logic, size recommendation logic, AI prompt formatting*
  - ***Execution:*** *PyTest / Jest executed on every git push*
  - ***Isolation:*** *Mock database and AI API*
  - ***Pass Condition:*** *Correct outputs for both normal and edge cases (eg. negative stock, missing size)*

- **Integration Test**
  - ***Scope:*** *Inventory service ↔ AI recommendation engine; Customer profile ↔ size matching*
  - ***Execution:*** *After merging feature branches*
  - ***Environment:*** *Real test database + AI API (low temperature setting)*
  - ***Pass Condition:*** *AI outputs must only include in-stock items with correct size filtering*

- **Test Environment (CI/CD Practice):**
  - ***Local:*** *Docker + SQLite for development testing*
  - ***CI/Staging:*** *GitHub Actions with PostgreSQL test database*
  - ***Pipeline Flow:*** *Lint → Unit Test → Integration Test → Deploy to staging*

- **Regression Testing & Pass/Fail Rules:**
  - ***Local:*** *Docker + SQLite for development testing*
  - ***CI/Staging:*** *GitHub Actions with PostgreSQL test database*
  - ***Pipeline Flow:*** *Lint → Unit Test → Integration Test → Deploy to staging*
  - *Continuation Rule: If any critical stock test fails → block deployment*

- **Test Data Strategy:**
  - ***Manual:*** *10 predefined clothing items (XS–XL, varied stock levels)*
  - ***Automated:*** *SQL seed scripts generating 100 random clothing items and 20 customer profiles*

- **Passing Rate Threshold:**
  - *A minimum of 86.2% of all test cases that are executed must pass. For the critical tests, all the stock correctness and size-filtered suggestions (100%) need to be passed, and there is no exception for that. Because a critical failure costs a huge amount in production*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***5 Errors***	***Failed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***97%***	***Failed***
Test Coverage	***Minimum 81.2%***	***79%***	***Failed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario that triggers an error in the system, and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*:*** *This test consists of non-feature-based tests, but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***End-to-End flow (stock + AI recommendation)***	*This test verifies the complete workflow where a customer purchases an item, and the system updates inventory while generating AI-based outfit*	*1. Customer purchases a clothing item (eg, red dress, size M)* *2. System updates stock*	*- Stock should be accurately reduced* *-AI suggests only in-stock items with correct sizes*	*Successfully ensures accurate and reliable system integration between inventory management*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		suggestions	*quantity* *3. User requests outfit recommendations*		*and AI recommendation.* *Status: Passed*
TC-02	***Negative Case (Size Filtering Validation)***	*Verify and ensure the system does not recommend items that are unavailable in the user's size.*	*1. Set customer size to XL* *2. Ensure only smaller sizes (S/M) are available* *3. Request recommendations*	*System returns "No items available in your size" or suggests alternatives, hence no incorrect size recommendations are generated*	*AI recommends an item in size M and claims it "fits relaxed". Size filtering is ignored, resulting in an inaccurate suggestion* *Status: Failed*
TC-03	***Performance Test (Concurrent Stock Updates)***	*Evaluates the system's ability to handle multiple simultaneous stock updates without causing data inconsistency or race conditions.*	*1. Simulate 50 concurrent users updating the same item's stock* *2. Each user adds +1 to the stock quantity* *3. Monitor final stock value*	*- Final stock = Initial stock + 50* *-No lost updates or inconsistencies* *-System maintains data integrity under concurrent operations*	*-Final stock is lower than expected (eg, Initial 10 → Final 58 instead of 60)* *-Some updates are lost due to race conditions* *-Indicates a lack of proper concurrency control* *Status: Failed*

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, define the acceptance criteria - what a correct, acceptable, or failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Recommend an outfit with a blue denim jacket (size L, stock=3) and suggest 2 additional items from leftover stock."	-Includes the denim jacket -Suggests 2 matching items that are **in stock and size-compatible** -Clearly states item name, size, and availability	-Denim jacket (stock 3) -White T-shirt (stock 0) -Black jeans (stock 2)	Failed
AI-02	"My chest is 38 inches, height is 170 cm. What size should I choose?"	-Recommends size **L (or M–L range)** -Includes explanation (e.g., "L fits 36–39 inches")	"Size L is perfect for you." No explanation or size range provided	Failed
AI-03	"Show 3 ways to use leftover green fabric (2 meters)."	-Practical suggestions within a 2m limit -Example: scarf, tote bag, headband	-Scarf (1.2m) -Cushion cover (1.5m) -Face mask (0.3m)	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

In this section, define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	1500 tokens
Input used while testing	4500 words
Expected Behavior	System rejects input Displays: *"Input too long. Please limit to 800 words."*
Actual Behavior	System silently truncates input Returns incomplete and inaccurate suggestions
Status	Failed

6.3. Adversarial/Edge Prompt Test

In this section, test a minimum of 1 with one prompt that is ambiguous, malformed - document how GLM responds and how your system is going to handle it.

PI-01 Prompt:
"I need elephant-sized clothes. Show me options."
Expected Output:

- Reject invalid size
- Return: *"Size not recognized. Please select XS–XL."*

Actual Output:

- "Elephant size not available, try XXL."

Status: Failed

6.4. Hallucinating Handling

© UMHackathon 2026

Email: umhackathon@um.edu.my

This section is defined for you to describe how your system detects or mitigates hallucinating responses.

Important Note:

Using Agile principles, these tests will take place iteratively over the period of development, not limited to the end.

Example case:

Expected Behavior:

- *AI suggestions must:*
  - *Match real inventory*
  - *Only include available stock and correct sizes*
- *Invalid outputs should be:*
  - *Rejected*
  - *Re-generated*
  - *Or replaced with a fallback message*

Actual Behavior:

- *AI sometimes:*
  - *Suggests out-of-stock items*
  - *Ignores size constraints*
  - *Provides vague or incomplete explanations*

1. Hallucination Detection Mechanisms

The system detects potential hallucinations using the following approaches:

- ***Data Grounding Validation***
  All AI-generated outputs are cross-checked against actual inventory and sales data. For example, outfit recommendations must only include items that exist in the database.
- ***Rule-Based Constraints***
  Logical constraints are applied to detect invalid outputs, such as:
  - *Recommending out-of-stock items*
  - *Pairing incompatible clothing categories (e.g., a winter coat with summer shorts in an irrelevant context)*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- ***Confidence & Consistency Checks***
  Responses are evaluated for consistency with previous outputs and input prompts. Sudden contradictory recommendations are flagged as potential hallucinations.
- ***Prompt Scope Validation***
  If the AI response deviates from the user's query (off-topic answers), it is flagged for correction.

2. Hallucination Mitigation Strategies

Once detected, the system applies the following mitigation strategies:

- ***Self-Correction Loop (Retry Mechanism)***
  The system automatically re-prompts the AI with stricter constraints to regenerate a more accurate response.
- ***Simplified Insight Fallback***
  If repeated attempts fail, the system returns a simplified, data-driven insight (e.g., "Item is slow-moving based on sales data") instead of speculative recommendations.
- ***Explainability Layer***
  Each AI output includes a brief explanation (e.g., "Recommended due to high demand trend"), allowing users to verify reasoning and identify possible inaccuracies.
- ***Human-in-the-Loop Validation***
  Users (retailers) are encouraged to review AI suggestions before execution to ensure final decisions remain reliable.

3. Agile Testing Approach

In alignment with Agile principles, hallucination testing is conducted ***iteratively throughout the development cycle****, not only at the final stage:*

- ***Early Stage (Day 3–5):***
  Validate the correctness of basic inventory insights and rule-based outputs
- ***Mid Stage (Day 6–7):***
  Test AI-generated recommendations for accuracy and relevance
- ***Late Stage (Day 8–9):***
  Perform integration testing to ensure consistency across frontend, backend, and AI components
- ***Continuous Testing:***
  Each sprint iteration includes prompt testing, edge case scenarios, and validation against real/simulated data

4. Example Test Scenario

© UMHackathon 2026

Email: umhackathon@um.edu.my

- ***Input:*** *"Suggest outfits to sell leftover summer skirts."*
- ***Expected Output:*** *Only valid inventory items paired with seasonally relevant clothing*
- ***Hallucination Case:*** *AI suggests non-existent items or irrelevant winter wear*
- ***Mitigation:*** *System retries with constraints → fallback to simplified recommendation if needed*

© UMHackathon 2026

Email: umhackathon@um.edu.my